

Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Estado de México, Escuela de Ingeniería y Ciencias.

Unidad de formación:

TC3009C.601

Inteligencia artificial avanzada para la ciencia de datos.

Actividad M2: Evaluación de Modelos

Equipo 5

Emiliano Caballero Mendoza A01749050

Índice

1. [Repositorio de GitHub](#)
 2. [Implementación del Algoritmo](#)
 3. [Pruebas y Predicciones](#)
 4. [Entorno de Ejecución](#)
 5. [Resultados y Análisis de Desempeño](#)
 - 5.1 [Datasets de Entrenamiento y Prueba](#)
 - 5.2 [Matriz de Confusión y Métricas](#)
 - 5.3 [Análisis y Conclusión](#)
 6. [Correcciones y Retroalimentación](#)
-

1. Repositorio de GitHub

- URL del Repositorio: <https://github.com/Wolv2/perceptron-training>

2. Implementación del Algoritmo

Descripción del algoritmo seleccionado y detalles sobre su programación manual.

- **Algoritmo implementado:** Perceptron
- **Descripción técnica:** El perceptrón es un modelo de clasificación binaria que aprende a separar dos clases mediante un hiperplano lineal. Su implementación consta de un **vector de pesos (w)** y un término de sesgo o **bias (b)**, ambos inicializados en cero. Para cada muestra de entrada, el modelo calcula una combinación lineal de las características ponderadas por los pesos más el bias

$$w \cdot x + b$$

Este valor se pasa por una función de activación tipo escalón (*step function*): si el resultado es mayor o igual a cero, la predicción es **1**; en caso contrario, es **0**. Aunque en la formulación clásica del perceptrón (Rosenblatt, 1958) suelen emplearse etiquetas **$\{-1, 1\}$** , en esta implementación se utilizó la convención **$\{0, 1\}$** , más común en aplicaciones actuales de clasificación binaria. El aprendizaje ocurre de forma iterativa: por cada muestra procesada

dentro de una época, se calcula el error como la diferencia entre la etiqueta real y la predicción

$$(\text{error} = y - y_{\text{pred}})$$

Si existe error, los pesos y el bias se actualizan de inmediato (no al finalizar la época completa) siguiendo la regla de entrenamiento del perceptrón:

$$\Delta w_i = \eta \cdot \text{error} \cdot x_i \quad \Delta b = \eta \cdot \text{error}$$

Donde η (**learning rate**) controla la magnitud del ajuste en cada actualización. Este proceso se repite durante un número fijo de épocas (criterio de paro utilizado en esta implementación), llevando además un conteo de errores por época para analizar visualmente la evolución y desempeño del modelo durante el entrenamiento. Dado que no se permitió el uso de librerías de aprendizaje automático, tanto la función de activación como la regla de actualización de pesos fueron implementadas manualmente utilizando únicamente NumPy para operaciones vectoriales básicas (producto punto). (*Perceptron Algorithm for Classification Using Sklearn*, n.d.)

3. Pruebas y Predicciones

- **Set de datos utilizado:** **Mobile Price Classification** (*Mobile Price Classification*, n.d.)
- **Método de visualización:** Consola
- **Ejemplos de predicciones:**

A continuación se muestran 8 ejemplos de predicciones generadas por el modelo entrenado, tomados del conjunto de prueba (datos no vistos durante el entrenamiento). Cada fila representa un celular con sus características originales sin normalizar: **battery_power** (capacidad de batería en mAh), **ram** (memoria RAM en MB), **int_memory** (memoria interna en GB) y **touch_screen** (1 = cuenta con pantalla táctil, 0 = no cuenta). La columna **Predicción** indica la clase asignada por el perceptrón, y **Valor Real** es la clase verdadera registrada en el dataset, donde **1 = celular caro** (originalmente **price_range** 2 o 3) y **0 = celular barato** (originalmente **price_range** 0 o 1).

battery_power	ram	int_memory	touch_screen	Predicción	Valor Real
1965	3535	4	1	1	1
790	3169	11	0	1	1
1426	1277	12	0	0	0
930	2457	14	1	1	0
1584	1561	41	0	0	0
1135	3029	46	1	1	1
965	1138	13	1	0	0
1983	2775	43	1	1	1

Tabla 1. Ejemplo de predicciones

De los 8 ejemplos mostrados, el modelo acertó en 7 de ellos (**87.5%**), un resultado congruente con el desempeño general obtenido sobre el conjunto completo de prueba.

Se destaca el único caso de error en esta muestra (fila con battery_power=930, ram=2457): el modelo predijo "caro" cuando el valor real era "barato". Una posible explicación es que el valor de RAM (2457 MB) es relativamente elevado en comparación con otros ejemplos correctamente clasificados como "barato" en este mismo conjunto (por ejemplo, ram=1277 o ram=1138), lo que sugiere que estas cuatro características, si bien son buenas predictoras en la mayoría de los casos, no bastan por sí solas para discriminar perfectamente entre ambas clases en todos los escenarios. Esto es consistente con la naturaleza lineal del perceptrón: al no poder capturar interacciones más complejas entre variables (como las que sí podría modelar una red neuronal multicapa), existen casos límite donde el modelo se equivoca.

4. Entorno de Ejecución

Confirmación de la portabilidad del código.

- **Lenguaje de programación:** Python
- **Archivo principal:** perceptron.py

- **Instrucciones de compilación/ejecución:** Ir a la carpeta perceptron-training desde tu consola, una vez estando en la carpeta correr el siguiente comando.
py [perceptron.py](#)

```
2\perceptron-training> py perceptron.py
```

5. Resultados y Análisis de Desempeño

5.1 Datasets de Entrenamiento y Prueba

Identificación de los conjuntos de datos utilizados para el desarrollo del modelo.

- **Dataset de entrenamiento:** subconjunto del 80% de **train.csv** (Mobile Price Classification, Kaggle), obtenido mediante partición aleatoria manual.
- **Dataset de prueba:** el 20% restante del mismo **train.csv**, no visto durante el entrenamiento.

Se optó por usar solamente **train.csv** y no **test.csv**, debido a que este último no incluye la columna **price_range**, por lo que no sería posible calcular el accuracy ni ninguna otra métrica de evaluación del modelo sobre ese archivo.

El atributo objetivo **price_range** presentaba originalmente cuatro niveles de clasificación (0: económico, 1: intermedio, 2: elevado y 3: muy elevado). Dado que el perceptrón es un modelo de clasificación binaria, se realizó un proceso de binarización de etiquetas: los dispositivos con categorías 0 y 1 fueron reasignados al valor **0 (celular "barato")**, mientras que aquellos pertenecientes a los rangos 2 y 3 recibieron la etiqueta **1 (celular "caro")**. Esta transformación se efectuó aplicando el criterio de decisión **price_range >= 2**. Esta misma convención (0 = barato, 1 = caro) se utiliza de forma consistente a lo largo de todo el documento, incluyendo la tabla de ejemplos de predicciones (sección 3) y la matriz de confusión (sección 5.2).

5.2 Matriz de Confusión y Métricas

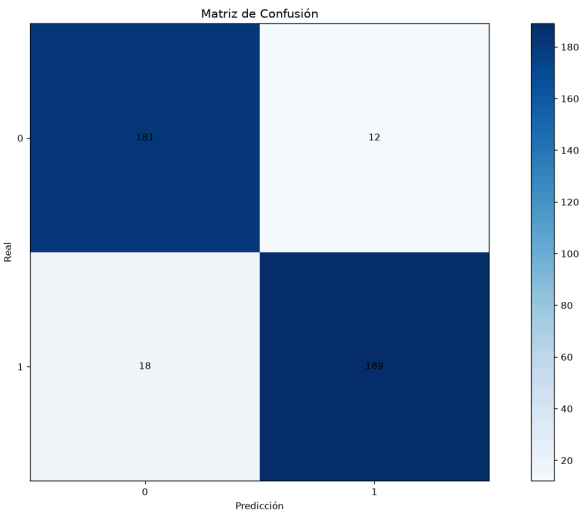


Fig 1. Matriz de Confusión

Métrica	Valor Obtenido
Accuracy	0.895
Precision	0.8857142857142857
Recall	0.9117647058823529
F1-Score	0.8985507246376812

Tabla 2. Metricas: Accuracy, Precision, Recall, F1-Score

5.3 Análisis y Conclusión

El desempeño del algoritmo implementado manualmente fue satisfactorio: como se observa en las métricas obtenidas, la implementación alcanzó valores cercanos al **90%** en **accuracy, precision, recall y F1-score**. Esto sugiere que, para este problema específico, el modelo resulta útil y su desempeño puede considerarse confiable.

Con estos resultados se demuestra que es posible predecir, con un alto grado de acierto, si un celular será clasificado como "caro" o "barato" a partir de un conjunto reducido de características (batería, RAM, memoria interna y pantalla táctil).

Sin embargo, es importante señalar las limitaciones inherentes al modelo. Al tratarse de una arquitectura de una sola capa, el perceptrón únicamente puede aprender fronteras de decisión lineales, lo cual le impide alcanzar un desempeño perfecto en problemas donde las clases no son completamente separables por una línea recta o hiperplano.

Además, el modelo está diseñado exclusivamente para clasificación binaria (verdadero/falso), por lo que no es posible utilizarlo directamente para predecir valores numéricos continuos (por ejemplo, el precio exacto de un celular) ni para problemas de clasificación con más de dos clases, sin antes adaptar su arquitectura o su función de salida.

A pesar de estas limitaciones, el perceptrón representa la base conceptual sobre la cual se construyen modelos más complejos, como las redes neuronales multicapa (backpropagation), las cuales superan estas restricciones al incorporar capas ocultas y funciones de activación no lineales. Esto convierte al perceptrón en un punto de partida fundamental para comprender el funcionamiento de arquitecturas más sofisticadas de aprendizaje automático.

6. Referencias

Mobile Price Classification. (n.d.). Kaggle. Retrieved September 1, 2026, from <https://www.kaggle.com/datasets/iabhishekoofficial/mobile-price-classification>

Perceptron Algorithm for Classification using Sklearn. (n.d.). GeeksforGeeks. Retrieved September 1, 2026, from <https://www.geeksforgeeks.org/deep-learning/sklearn-classification-using-perceptron/>